



School of Future Tech

Case Study Report

Cloud Resource Allocation Optimization

Your Case Study Title

by

Ankitraj jha, Sejal Singh, Sairaj Jadhav, Parthiv Kumar

150096725050, 150096725045 , 150096725067, 150096725095

Index

1. Introduction to the Case Study.
2. Problem Statement / Case Background (Abstract).
3. Problem Statement / Case Study Design.
4. Methods & Algorithms Technology Applied in the Problem Statement / Case Study.
5. Problem Statement / Case Study Implementation Details and Snapshots.
6. Problem Statement / Case Study Results and Conclusion.
7. References

1. Introduction to the Case Study

Cloud computing systems are designed to handle large-scale workloads efficiently across multiple servers. With the rapid growth of modern applications such as e-commerce platforms, streaming services, and AI-driven systems, the demand for efficient resource utilization has significantly increased.

In such environments, improper distribution of tasks can lead to server overload, increased latency, and degraded system performance. Therefore, intelligent task allocation strategies are essential to ensure smooth system operation.

This case study focuses on optimizing the allocation of computational tasks across available servers using an algorithmic approach. The primary goal is to distribute workloads in such a way that no single server becomes a bottleneck, thereby improving system efficiency, reliability, and response time.

Efficient resource allocation is widely applicable in:

- Cloud service providers (AWS, Azure, GCP)
- Data centers
- Distributed computing systems
- Parallel processing environments

2. Problem Statement (Abstract)

In large-scale distributed systems, multiple tasks with varying computational loads must be assigned to a limited number of servers.

The main challenges include:

- Uneven distribution of workloads
- Server overloading
- Increased response time due to bottlenecks

The objective of this case study is to:

- Distribute tasks among servers efficiently
- Ensure that the **maximum load on any server is minimized**
- Maintain overall system performance and scalability

This problem is a variation of the **load balancing problem**, closely related to:

- Partition problems
- Job scheduling problems
- Resource allocation optimization

Efficient solutions to this problem directly contribute to improved system throughput and reduced operational costs.

3. Case Study Design

Input:

- A list of tasks, where each task has a specific load value
- A fixed number of servers available for processing

Constraints:

- Each task must be assigned to exactly one server
- Tasks must be allocated in sequential order (no reordering allowed)
- Each server can process multiple tasks
- Number of servers must be $\leq$ number of tasks

Objective:

Minimize the **maximum load assigned to any server**, ensuring balanced utilization.

Real-World Interpretation:

- Tasks $\rightarrow$ User requests / jobs / processes
 - Servers $\rightarrow$ Machines / CPUs / containers
 - Load $\rightarrow$ CPU usage / memory usage / execution time
-

Example:

Tasks = {10, 20, 30, 40}

Servers = 2

Possible distribution:

- Server 1 $\rightarrow$ {10, 20, 30} $\rightarrow$ Load = 60

- Server 2 $\rightarrow \{40\} \rightarrow \text{Load} = 40$

Optimal maximum load = **60**

Why This Problem Matters:

- Prevents server crashes due to overload
- Improves response time
- Ensures fairness in workload distribution
- Enhances scalability in cloud systems

4. Methods & Algorithms Technology Applied

Algorithm Used: Binary Search on Answer

Instead of checking all possible ways to distribute tasks (which is computationally expensive), we use an optimized approach called **Binary Search on Answer**.

Why Binary Search Works Here:

The answer (maximum load) lies within a range:

- Minimum possible = largest single task
- Maximum possible = sum of all tasks

This creates a **monotonic search space**, making it ideal for binary search.

Step-by-Step Approach:

Step 1: Define Search Space

- `low = max(task)`
- `high = sum(task)`

Step 2: Apply Binary Search

- Calculate mid value
- Assume mid is the maximum allowed load

Step 3: Feasibility Check (Greedy Approach)

- Assign tasks sequentially
- If adding a task exceeds mid $\rightarrow$ allocate new server
- Count total servers required

Step 4: Decision Making

- If required servers $\leq$ available $\rightarrow$ valid
 - Else $\rightarrow$ increase load
-

Greedy Strategy Explanation:

The algorithm uses a greedy approach by:

- Filling one server as much as possible
- Moving to the next only when necessary

This ensures:

- Minimum number of servers used for a given load
- Efficient validation of feasibility

Time Complexity Analysis:

Component	Complexity
Binary Search	$O(\log(\text{Sum} - \text{Max}))$
Feasibility Check	$O(N)$
Overall	$O(N \log N)$

Space Complexity:

- $O(N)$ for storing tasks
- $O(N)$ for allocation result

Technologies Used:

- **C++ STL**
 - `vector` for dynamic arrays
 - `accumulate` for sum calculation
 - `max_element` for finding max task
 - **Greedy Algorithm**
 - **Binary Search Optimization**
-

5. Implementation Details and Snapshots

Core Functions Description:

1. `isPossible()`

- Checks whether a given maximum load is feasible
- Uses greedy allocation strategy
- Returns true if allocation is possible within given servers

2. `optimizeCloudResources()`

- Performs binary search on load values
- Determines optimal maximum load
- Reconstructs final task allocation

Important Implementation Features:

- ✓ Uses `long long` to handle large inputs and prevent overflow
- ✓ Efficient memory usage (no repeated allocation in loop)
- ✓ Fast I/O improves performance for large datasets
- ✓ Clean modular design with separate functions

Edge Case Handling:

- Invalid number of servers

- Single task scenario
- Large input values
- Equal task loads

Sample Input:

Tasks: 5

Loads: 10 20 30 40 50

Servers: 3

Sample Output:

Optimal Maximum Load Value: 60

Server 1 [Load: 60] → {10, 20, 30}

Server 2 [Load: 40] → {40}

Server 3 [Load: 50] → {50}

Interpretation of Output:

- No server exceeds load 60
- Tasks are distributed efficiently
- Load difference between servers is minimized

6. Results and Conclusion

Results:

- Achieved optimal load balancing across servers
- Reduced the maximum load per server
- Ensured efficient utilization of resources
- Improved execution efficiency for large-scale inputs

Performance Benefits:

- Avoids brute-force exponential solutions
- Works efficiently even for large datasets
- Scalable for real-world cloud environments

Conclusion:

The **Binary Search on Answer** technique provides a highly efficient and scalable solution for cloud resource allocation problems. By combining binary search with a greedy feasibility check, the algorithm ensures optimal workload distribution with minimal computational overhead.

This approach:

- Enhances system performance
- Reduces latency
- Improves scalability
- Prevents server overloading

Real-World Applications:

- Cloud computing platforms (task scheduling)
- Distributed systems
- Load balancers
- Parallel computing systems
- Microservices architecture

7. References

1. Cormen, T. H., et al. — *Introduction to Algorithms*
2. GeeksforGeeks — Load Balancing & Binary Search Problems
3. LeetCode — Split Array Largest Sum Problem
4. C++ STL Documentation — Standard Template Library
5. Research Papers on:
 - Distributed Systems Optimization
 - Cloud Resource Scheduling
 - Load Balancing Techniques

